

Be Cautious While Using Bit Fields for Programming

In this article, the author tells embedded C programmers why writing code involving bit fields needs to be done carefully, failing which the results may not be as expected. However, bit fields are handy as they involve only low level programming and result in efficient data storage.



Embedded C programmers who have worked with structures should also be familiar with the use of bit fields structures. Use of bit fields is one of the key optimisation methods in embedded C programming, because these allow one to pack together several related entities, where each set of bits and single bits can be addressed. Of course, the usage of bit fields is 'easy' and comes handy, especially during low level programming. Though considered as one of the unique features of C programming, bit fields do have some limitations. Let us look at these by exploring the example problems in this article.

Data types and bit fields

Let us look into the signed qualifiers affecting the output of the bit field structure. Please note that the code snippets provided here are tested with the GCC compiler [gcc version 4.7.3] running under a Linux environment.

Let us consider a simple small C code snippet as shown below, with a structure named *bit field*, with three integer fields—*hours*, *mins* and *secs*, of bit field sizes 5, 6 and 6, respectively:

```
typedef struct bit_field
{
    int hours : 5;
    int mins : 6;
    int secs : 6;
}time_t;
```

Now let us declare a variable *alarm* of type *time_t* and set values as 22, 12 and 20, respectively:

```
//Declaration of the variable of type time_t
time_t alarm;
//Assigning the values to the different members of the bit-
field structures
alarm.hours = 22;
alarm.mins = 12;
alarm.secs = 20;
```

When we print these values using a simple *printf* statement, what could be the output? At first, most of us will envision the answers to be 22, 12 and 20, for *hours*, *mins* and *secs* respectively. Whereas when we actually compile and run the code, the value printed for the hours would be different - 10, 12 and 20 (as shown in Figure 1).

Where did we go wrong?

1. We all know that the default signed qualifier for the 'int' is 'signed int'.
2. We reserved 5 bits for storing the *hours* field assuming we were using the 24-hour format. From among 5 bits, 1 bit was used for storing the sign of the number, which means only 4 bits were then available for storing the actual value. In these 4 bits, we can store the numbers ranging from -16 to +15 according to the formula (-2^k) to $(+2^k - 1)$ including '0', where 'k' indicates the number of bits.